DEFENSECLAW

FEATURE RELEASE

Capability-Based Access Control

Granular, runtime-enforced access control for AI agents with scan-driven automatic policy generation, default-deny evaluation, and constraint matching

PHASE 1

Capability Evaluator

Default-deny, constraints, restrictions, conditions

PHASE 2

Auto-Generation

Scan-informed policies, manifest introspection

April 2026 | v1.0 | Enterprise Governance Layer for AI Agents

1. Executive Overview

DefenseClaw now provides **capability-based access control** for AI agents — a granular, runtime-enforced system that goes far beyond binary block/allow decisions. Instead of giving an agent full access to an MCP server or none at all, capabilities define *exactly* what each agent can do, with constraints, restrictions, time windows, and rate limits.

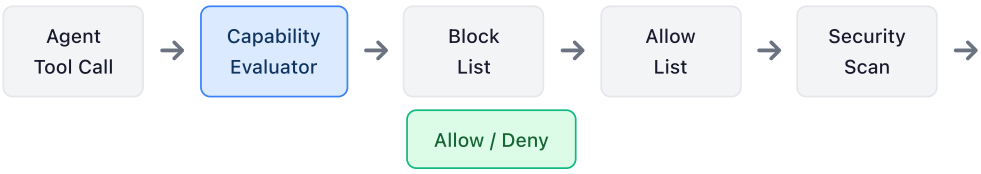
The system is built in two phases: **Phase 1** delivers the evaluator engine, policy format, CLI commands, TUI integration, and audit logging. **Phase 2** adds automatic policy generation that introspects MCP/skill manifests and adapts security posture based on scan findings.

6	3	5	15	21+
EVAL STEPS	POSTURES	CLI COMMANDS	SOURCE FILES	UNIT TESTS

2. Architecture

Enforcement Flow (Phase 1)

Capability evaluation is the **first check** before the existing admission gate. If denied, the tool call never reaches the scan pipeline:

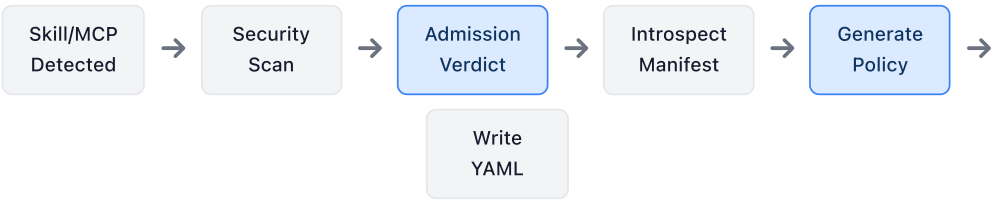


Deny at capability check → short-circuit (no scan needed)

6-Step Evaluation Pipeline

#	Step	Action	Deny Reason
1	Load Agent Policy	Find `.capability.yaml` for agent	`unknown agent`
2	Check Restrictions	Hard blocks (no_write, no_delete, etc.)	`restricted`
3	Check Conditions	Time window, environment, rate limit	`condition: <which>`
4	Match Capabilities	Find capability matching resource	`no capability for resource`
5	Evaluate Constraints	Glob, list membership, range checks	`constraint mismatch`
6	Log Decision	Write to SQLite audit store	—

Auto-Generation Flow (Phase 2)



Blocked/rejected → no policy generated | Clean/warning/allowed → auto-generate

3. Phase 1: Capability Evaluator

Policy Manifest Format

Each agent gets a `.capability.yaml` file in `~/.defenseclaw/capabilities/`. Policies express *what* an agent can do with fine-grained constraints:

support-bot.capability.yaml

```yaml
agent: support-bot
description: "Customer support automation agent"

capabilities:
  - name: read_jira_ticket
    resource: "jira.get_issue"
    constraints:
      project: "ENG-*"                  # glob pattern
      fields: [summary, status, assignee]  # list membership

  - name: post_slack_message
    resource: "slack.post_message"
    constraints:
      channel: "#support"
      template: approved_templates_only

  - name: read_confluence
    resource: "confluence.get_page"
    constraints:
      space: SUPPORT

restrictions:
  - "no_external_http"
  - "no_bulk_export"

conditions:
  time_window: "09:00-18:00"           # UTC
  environments: [production, staging]
  rate_limit:
    max_calls: 100
    window_seconds: 3600
```

Constraint Matching & Restrictions

String Values — Glob Patterns

Uses `filepath.Match` for glob matching. `project: "ENG-*"` allows `ENG-123` but denies `FIN-456`.

Slice Values — List Membership

All requested fields must be in the allowed list. `fields: [summary, status]` allows requesting `summary` but denies `comments`.

Restrictions — Hard Blocks

Checked *before* capabilities. No capability can override a restriction.

Restriction	Blocks
`no_external_http`	`http.*`, `external_http.*`
`no_bulk_export`	`*.export_all`, `*.bulk_export`
`no_write`	`*.create_*`, `*.update_*`, `*.post_*`
`no_delete`	`*.delete_*`, `*.remove_*`

Conditions (Agent-Global)

Time Window

UTC-based, supports midnight crossing (e.g., `22:00-06:00`). Calls outside the window are denied.

Environment

Restricts agent to named environments. Empty list allows all.

Rate Limiting

Sliding window tracked in SQLite `capability_calls` table. Exceeding the limit → deny until window clears.

Default-Deny

Unknown agents, unmatched resources, and failed constraints all result in denial.

> **Key principle:** Capabilities are not tools — they are *constrained permissions*. An agent may have access to `jira.get_issue` but only for project `ENG-*` during business hours with a 100-call/hour limit.

4. Phase 1 Demo: CLI Commands & Evaluation

CLI: List Loaded Policies

Terminal — defenseclaw capability list

```
$ defenseclaw capability list

Agent Capability Policies (/Users/nghodki/.defenseclaw/capabilities)
──────────────────────────────────────────────────────────────────
  alert-service        3 capabilities, 0 restrictions
  default-agent        7 capabilities, 0 restrictions
```

CLI: Show Policy Details

Terminal — defenseclaw capability show

```
$ defenseclaw capability show alert-service

{
  "Agent": "alert-service",
  "Description": "Auto-generated from MCP scan (permissive)",
  "Generated": true,
  "Approved": true,
  "Capabilities": [
    { "Name": "get_weather",  "Resource": "alert-service.get_weather" },
    { "Name": "create_alert", "Resource": "alert-service.create_alert" },
    { "Name": "delete_alert", "Resource": "alert-service.delete_alert" }
  ],
  "Restrictions": [],
  "Conditions": {}
}
```

CLI: Evaluate — Allowed

Terminal — Evaluate (Allowed)

```
$ defenseclaw capability evaluate alert-service alert-service.get_weather

{
  "allowed": true,
  "reason": "capability matched",
  "capability": "get_weather"
}
```

CLI: Evaluate — Denied (no matching capability)

Terminal — Evaluate (Denied)

```
$ defenseclaw capability evaluate alert-service jira.get_issue

{
  "allowed": false,
  "reason": "no capability for resource"
}
```

CLI: Validate Policy File

Terminal — defenseclaw capability validate

```
$ defenseclaw capability validate ~/.defenseclaw/capabilities/alert-service.capability.yaml
Valid: alert-service (3 capabilities, 0 restrictions)
```

Audit Trail

Every evaluation decision is logged to the SQLite audit store with timestamp, agent, resource, parameters, allowed/denied, and reason. Rate limit calls are tracked in a separate `capability_calls` table with indexed queries for efficient sliding-window calculations.

> **5 CLI commands:** `list`, `show`, `evaluate`, `validate`, and `approve` (Phase 2). All follow existing Cobra patterns. Decisions are also forwarded to Splunk HEC and OTel.

5. Phase 2: Scan-Driven Auto-Generation

Without auto-generation, users must manually write `.capability.yaml` files for every agent. Phase 2 solves this by **introspecting manifests** for per-tool granularity and **adapting security posture based on scan findings**.

MCP Manifest Introspection

Parses the MCP server's JSON manifest to discover per-tool metadata:

Terminal — MCP Introspection

```
=== Step 1: MCP Manifest Introspection ===
Parsed 3 tools from alert-service MCP manifest:
  [tool] get_weather      Get current weather for a location
  [tool] create_alert     Create a new alert rule
  [tool] delete_alert     Delete an existing alert
```

Skill Manifest Introspection

Terminal — Skill Introspection

```
=== Step 2: Skill Manifest Introspection ===
Parsed skill: permissioned-skill
  Permissions: [read-only]
```

Scan-Informed Posture Matrix

Scan Result	Posture	Capabilities	Restrictions	Rate Limit
Clean No findings	**Permissive**	All discovered tools	None	None
LOW / MEDIUM	**Cautious**	All discovered tools	no_bulk_export	100 / hour
HIGH / CRITICAL	**Restrictive**	Read-like tools only	no_write, no_delete, no_bulk_export	50 / hour

Policy Generation Demo — Three Postures

Permissive (Clean Scan)

```
Permissive (clean scan)
  Agent:        auto-alert-service
  Generated:    true
  Approved:     false
  Capabilities: 3
    get_weather    -> alert-service.get_weather
    create_alert   -> alert-service.create_alert
    delete_alert   -> alert-service.delete_alert
  Restrictions: (none)
```

Cautious (MEDIUM Findings)

```
Cautious (MEDIUM findings)
  Agent:        auto-data-service
  Capabilities: 3
  Restrictions: [no_bulk_export]
  Rate Limit:   100 calls / 3600s
```

Restrictive (HIGH Findings)

```
Restrictive (HIGH findings)
  Agent:        auto-risky-service
  Capabilities: 1 (read-like only)
    get_weather    -> risky-service.get_weather
  Restrictions: [no_write no_delete no_bulk_export]
  Rate Limit:   50 calls / 3600s
```

Skill Policy with read-only Permission

Terminal — Skill Policy Generation

```
=== Skill Policy (read-only permissions) ===
  Agent:        auto-permissioned-skill
  Capabilities: 1
    permissioned-skill -> permissioned-skill.*
  Restrictions: [no_write no_delete]
```

Read-like detection: Tools with prefixes `get_`, `list_`, `read_`, `search_`, `fetch_`, `query_`, `describe_`, `show_` are considered safe for the restrictive posture. All other tools are filtered out when HIGH/CRITICAL findings exist.

6. Phase 2 Demo: Approval Workflow & TUI

Approval Flow

Auto-generated policies are active immediately but flagged as `pending review`. The approve command promotes ownership to the user:

CLI: Approve Auto-Generated Policy

Terminal — defenseclaw capability approve

```
$ defenseclaw capability approve alert-service
Approved: alert-service (3 capabilities, 0 restrictions)

$ defenseclaw capability list

Agent Capability Policies (/Users/nghodki/.defenseclaw/capabilities)
──────────────────────────────────────────────────────────────
  alert-service        3 capabilities, 0 restrictions
  default-agent        7 capabilities, 0 restrictions
```

Error Handling

Terminal — Error Cases

```
$ defenseclaw capability approve nonexistent
Error: capability: no pending auto-generated policy for "nonexistent"

$ defenseclaw capability approve alert-service  # already approved
Error: capability: manual policy already exists — edit it directly
```

TUI: Agents Panel with Status Column

The TUI "Agents" tab shows policy status with color coding — `pending review` for unapproved auto-generated policies, `approved` or `manual` otherwise:

TUI — Agents Panel

```
Agents                                          Status
──────────────────────────────────────────────────────────
alert-service      3 caps, 0 restrictions       approved
default-agent      7 caps, 0 restrictions       manual
data-service       3 caps, 1 restriction        pending review
risky-service      1 caps, 3 restrictions       pending review
```

Watcher Integration

Auto-generation hooks directly into `runAdmission()` at 5 return points (allowed/clean/warning verdicts). No new goroutines or services — it runs inline after the admission verdict.

> **Conflict resolution:**
>
> - **Manual policy exists** → skip auto-generation entirely (manual always wins)
> - **Auto policy already exists** → skip (don't overwrite on re-scan)
> - **Introspection fails** → fall back to wildcard `<name>.*` capability
> - **Approve when manual exists** → error (user should edit manual file)

7. Test Suite

Terminal — go test -v (auto-generation tests)

```
$ go test ./test/unit/ -run "TestAgentPolicy|TestIntrospect|TestGeneratePolicy|TestApprovePoli

--- PASS: TestApprovePolicySuccess (0.00s)
--- PASS: TestApprovePolicyNoAutoFile (0.00s)
--- PASS: TestApprovePolicyManualExists (0.00s)
--- PASS: TestAgentPolicyGeneratedApprovedFields (0.00s)
--- PASS: TestAgentPolicyOmitEmptyGeneratedApproved (0.00s)
--- PASS: TestGeneratePolicyCleanMCP (0.00s)
--- PASS: TestGeneratePolicyCautiousMCP (0.00s)
--- PASS: TestGeneratePolicyRestrictiveMCP (0.00s)
--- PASS: TestGeneratePolicyFallbackWildcard (0.00s)
--- PASS: TestGeneratePolicySkillReadOnly (0.00s)
--- PASS: TestGeneratePolicySkillNoPermissions (0.00s)
--- PASS: TestGeneratePolicyNilScanResult (0.00s)
--- PASS: TestIntrospectMCP (0.00s)
    --- PASS: TestIntrospectMCP/3_tools (0.00s)
    --- PASS: TestIntrospectMCP/empty_tools_array (0.00s)
    --- PASS: TestIntrospectMCP/existing_clean_MCP_with_1_tool (0.00s)
    --- PASS: TestIntrospectMCP/missing_file (0.00s)
--- PASS: TestIntrospectMCPToolDetails (0.00s)
--- PASS: TestIntrospectMCPMalformedJSON (0.00s)
--- PASS: TestIntrospectSkill (0.00s)
--- PASS: TestIntrospectSkillNoPermissions (0.00s)
PASS
ok  github.com/defenseclaw/defenseclaw/test/unit
```

8. Implementation Summary

Phase 1 Files

File	Purpose
`capability/types.go`	Core types: AgentPolicy, Capability, Conditions, EvalRequest, Decision
`capability/evaluator.go`	6-step evaluator engine, hot-reload
`capability/constraints.go`	Glob, list, exact, range matching
`capability/loader.go`	YAML manifest loading + validation
`cli/capability.go`	list, show, evaluate, validate commands
`tui/agents.go`	Agents panel with decision history
`config/config.go`	CapabilityPolicyDir field
`audit/store.go`	capability_decisions + capability_calls tables

Phase 2 Files

File	Purpose
`capability/introspect.go`	MCP + skill manifest parsing
`capability/generator.go`	Scan-informed generation + approve
`watcher/watcher.go`	5 admission hooks for auto-gen
`gateway/sidecar.go`	Wire evaluator into watcher
`cli/capability.go`	approve subcommand
`tui/agents.go`	Status column + SetPolicies()
`tui/app.go`	Pass evaluator policies to panel

Key Design Decisions:

- **Default-deny** — unknown agents and unmatched resources are always rejected
- **Sidecar/hook mode** — no MCP proxy required, integrates with existing daemon
- **Scan-informed posture** — security adapts automatically to scan findings
- **No circular imports** — GenerateRequest uses primitive types + ScanResultSummary
- **Semi-blocking approval** — policies active immediately, flagged pending until approved
- **Graceful degradation** — introspection failures fall back to wildcard capabilities